Proyek Klasifikasi Gambar: Animals-10 Dataset

- **Nama:** Muh. Rinaldi Ruslan
- **Email:** rinaldi.ruslan51@gmail.com
- **ID Dicoding:** rinaldi51

Import Semua Packages/Library yang Digunakan

```
!pip install tensorflowjs
```

```
Requirement already satisfied: tensorflowjs in
/usr/local/lib/python3.12/dist-packages (4.22.0)
Requirement already satisfied: flax>=0.7.2 in
/usr/local/lib/python3.12/dist-packages (from tensorflowjs) (0.11.2)
Requirement already satisfied: importlib_resources>=5.9.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflowjs) (6.5.2)
Requirement already satisfied: jax>=0.4.13 in
/usr/local/lib/python3.12/dist-packages (from tensorflowjs) (0.7.2)
Requirement already satisfied: jaxlib>=0.4.13 in
/usr/local/lib/python3.12/dist-packages (from tensorflowjs) (0.7.2)
Requirement already satisfied: tensorflow<3,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflowjs) (2.19.0)
Requirement already satisfied: tf-keras>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflowjs) (2.19.0)
Requirement already satisfied: tensorflow-decision-forests>=1.5.0
in /usr/local/lib/python3.12/dist-packages (from tensorflowjs)
(1.12.0)
Requirement already satisfied: six<2,>=1.16.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflowjs) (1.17.0)
Requirement already satisfied: tensorflow-hub>=0.16.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflowjs) (0.16.1)
Requirement already satisfied: packaging~=23.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflowjs) (23.2)
Requirement already satisfied: numpy>=1.23.2 in
/usr/local/lib/python3.12/dist-packages (from flax>=0.7.2-
>tensorflowjs) (2.0.2)
Requirement already satisfied: msgpack in
/usr/local/lib/python3.12/dist-packages (from flax>=0.7.2-
>tensorflowjs) (1.1.2)
Requirement already satisfied: optax in
/usr/local/lib/python3.12/dist-packages (from flax>=0.7.2-
>tensorflowjs) (0.2.8)
Requirement already satisfied: orbax-checkpoint in
/usr/local/lib/python3.12/dist-packages (from flax>=0.7.2-
>tensorflowjs) (0.11.34)
Requirement already satisfied: tensorstore in
/usr/local/lib/python3.12/dist-packages (from flax>=0.7.2-
>tensorflowjs) (0.1.82)
```

```
Requirement already satisfied: rich>=11.1 in
/usr/local/lib/python3.12/dist-packages (from flax>=0.7.2-
>tensorflowjs) (13.9.4)
Requirement already satisfied: typing_extensions>=4.2 in
/usr/local/lib/python3.12/dist-packages (from flax>=0.7.2-
>tensorflowjs) (4.15.0)
Requirement already satisfied: PyYAML>=5.4.1 in
/usr/local/lib/python3.12/dist-packages (from flax>=0.7.2-
>tensorflowjs) (6.0.3)
Requirement already satisfied: treescope>=0.1.7 in
/usr/local/lib/python3.12/dist-packages (from flax>=0.7.2-
>tensorflowjs) (0.1.10)
Requirement already satisfied: ml_dtypes>=0.5.0 in
/usr/local/lib/python3.12/dist-packages (from jax>=0.4.13-
>tensorflowjs) (0.5.4)
Requirement already satisfied: opt_einsum in
/usr/local/lib/python3.12/dist-packages (from jax>=0.4.13-
>tensorflowjs) (3.4.0)
Requirement already satisfied: scipy>=1.13 in
/usr/local/lib/python3.12/dist-packages (from jax>=0.4.13-
>tensorflowjs) (1.16.3)
Requirement already satisfied: absl-py>=1.0.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1
in /usr/local/lib/python3.12/dist-packages (from
tensorflow<3,>=2.13.0->tensorflowjs) (0.7.0)
Requirement already satisfied: google-pasta>=0.1.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (18.1.1)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!
=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (2.32.4)
Requirement already satisfied: setuptools in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (75.2.0)
```

```
Requirement already satisfied: termcolor>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (3.3.0)
Requirement already satisfied: wrapt>=1.11.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (1.80.0)
Requirement already satisfied: tensorboard~=2.19.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (3.13.2)
Requirement already satisfied: h5py>=3.11.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow<3,>=2.13.0-
>tensorflowjs) (3.16.0)
Requirement already satisfied: pandas in
/usr/local/lib/python3.12/dist-packages (from tensorflow-decision-
forests>=1.5.0->tensorflowjs) (2.2.2)
Requirement already satisfied: wheel in
/usr/local/lib/python3.12/dist-packages (from tensorflow-decision-
forests>=1.5.0->tensorflowjs) (0.45.1)
Requirement already satisfied: wurlitzer in
/usr/local/lib/python3.12/dist-packages (from tensorflow-decision-
forests>=1.5.0->tensorflowjs) (3.1.1)
Requirement already satisfied: ydf>=0.11.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow-decision-
forests>=1.5.0->tensorflowjs) (0.15.0)
Requirement already satisfied: namex in
/usr/local/lib/python3.12/dist-packages (from keras>=3.5.0-
>tensorflow<3,>=2.13.0->tensorflowjs) (0.1.0)
Requirement already satisfied: optree in
/usr/local/lib/python3.12/dist-packages (from keras>=3.5.0-
>tensorflow<3,>=2.13.0->tensorflowjs) (0.19.0)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0-
>tensorflow<3,>=2.13.0->tensorflowjs) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0-
>tensorflow<3,>=2.13.0->tensorflowjs) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0-
>tensorflow<3,>=2.13.0->tensorflowjs) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0-
>tensorflow<3,>=2.13.0->tensorflowjs) (2026.2.25)
Requirement already satisfied: markdown-it-py>=2.2.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from rich>=11.1->flax>=0.7.2-
>tensorflowjs) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich>=11.1->flax>=0.7.2-
>tensorflowjs) (2.20.0)
Requirement already satisfied: markdown>=2.6.8 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0-
>tensorflow<3,>=2.13.0->tensorflowjs) (3.10.2)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0
in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0-
>tensorflow<3,>=2.13.0->tensorflowjs) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0-
>tensorflow<3,>=2.13.0->tensorflowjs) (3.1.8)
Requirement already satisfied: etils[epath,epy] in
/usr/local/lib/python3.12/dist-packages (from orbax-checkpoint-
>flax>=0.7.2->tensorflowjs) (1.14.0)
Requirement already satisfied: aiofiles in
/usr/local/lib/python3.12/dist-packages (from orbax-checkpoint-
>flax>=0.7.2->tensorflowjs) (24.1.0)
Requirement already satisfied: humanize in
/usr/local/lib/python3.12/dist-packages (from orbax-checkpoint-
>flax>=0.7.2->tensorflowjs) (4.15.0)
Requirement already satisfied: simplejson>=3.16.0 in
/usr/local/lib/python3.12/dist-packages (from orbax-checkpoint-
>flax>=0.7.2->tensorflowjs) (3.20.2)
Requirement already satisfied: psutil in
/usr/local/lib/python3.12/dist-packages (from orbax-checkpoint-
>flax>=0.7.2->tensorflowjs) (5.9.5)
Requirement already satisfied: uvloop in
/usr/local/lib/python3.12/dist-packages (from orbax-checkpoint-
>flax>=0.7.2->tensorflowjs) (0.22.1)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas->tensorflow-
decision-forests>=1.5.0->tensorflowjs) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas->tensorflow-
decision-forests>=1.5.0->tensorflowjs) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas->tensorflow-
decision-forests>=1.5.0->tensorflowjs) (2026.1)
Requirement already satisfied: mdurl~=0.1 in
/usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0-
>rich>=11.1->flax>=0.7.2->tensorflowjs) (0.1.2)
Requirement already satisfied: markupsafe>=2.1.1 in
/usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1-
>tensorboard~=2.19.0->tensorflow<3,>=2.13.0->tensorflowjs) (3.0.3)
Requirement already satisfied: fsspec in
/usr/local/lib/python3.12/dist-packages (from etils[epath,epy]->orbax-
```

```
checkpoint->flax>=0.7.2->tensorflowjs) (2025.3.0)
Requirement already satisfied: zipp in /usr/local/lib/python3.12/dist-
packages (from etils[epath,epy]->orbax-checkpoint->flax>=0.7.2-
>tensorflowjs) (3.23.0)
```

```
# BASIC LIBRARIES
import os
import numpy as np
import matplotlib.pyplot as plt
import shutil
import warnings

# IMAGE PROCESSING
from PIL import Image

# TENSORFLOW & KERAS
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau, ModelCheckpoint
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, Dropout,
GlobalAveragePooling2D, Conv2D, MaxPooling2D, Flatten,
BatchNormalization
from tensorflow.keras.models import Sequential

# DATA SPLITTING
from sklearn.model_selection import train_test_split

# SETTINGS
warnings.filterwarnings('ignore')

print("TensorFlow Version:", tf.__version__)

tf.random.set_seed(42)
np.random.seed(42)
```

```
TensorFlow Version: 2.19.0
```

Data Preparation

Data Loading

```
from google.colab import drive
drive.mount('/content/drive')

zip_path = "/content/drive/MyDrive/UNHAS/Chapter 6/Pijak/Belajar
Fundamental Deep Learning/Proyek Klasifikasi Gambar/Dataset/raw-
img.zip"
```

```python
print("Dataset Path:", zip_path)
```

```
Mounted at /content/drive
Dataset Path: /content/drive/MyDrive/UNHAS/Chapter 6/Pijak/Belajar
Fundamental Deep Learning/Proyek Klasifikasi Gambar/Dataset/raw-
img.zip
```

```python
# EXTRACT ZIP DATASET
import zipfile

extract_path = "/content/dataset"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset berhasil diekstrak ke:", extract_path)
```

```
Dataset berhasil diekstrak ke: /content/dataset
```

```python
# SET DATASET DIRECTORY
dataset_path = os.path.join(extract_path, "raw-img")

print("Dataset directory:", dataset_path)
```

```
Dataset directory: /content/dataset/raw-img
```

```python
# CHECK DATASET STRUCTURE
classes = os.listdir(dataset_path)

print("Jumlah kelas:", len(classes))
print("Nama kelas:", classes)
```

```
Jumlah kelas: 10
Nama kelas: ['pecora', 'gatto', 'cavallo', 'gallina', 'mucca',
'elefante', 'ragno', 'scoiattolo', 'cane', 'farfalla']
```

```python
# COUNT IMAGES PER CLASS
total_images = 0

for cls in classes:
    class_path = os.path.join(dataset_path, cls)
    num_images = len(os.listdir(class_path))
    print(f"{cls}: {num_images}")
    total_images += num_images

print("\nTotal gambar:", total_images)
```

```
pecora: 1820
gatto: 1668
cavallo: 2623
gallina: 3098
```

```
mucca: 1866
elefante: 1446
ragno: 4821
scoiattolo: 1862
cane: 4863
farfalla: 2112

Total gambar: 26179
```

```python
# CHECK IMAGE RESOLUTION VARIATION
unique_sizes = set()

for cls in classes:
    class_path = os.path.join(dataset_path, cls)
    for img_file in os.listdir(class_path)[:50]:
        img_path = os.path.join(class_path, img_file)
        try:
            with Image.open(img_path) as img:
                unique_sizes.add(img.size)
        except:
            continue

print("Contoh resolusi unik:", list(unique_sizes)[:10])
print("Jumlah variasi resolusi:", len(unique_sizes))
```

```
Contoh resolusi unik: [(300, 206), (256, 300), (300, 270), (243, 300),
(640, 448), (300, 236), (300, 300), (300, 245), (300, 190), (194,
300)]
Jumlah variasi resolusi: 162
```

```python
# SHOW SAMPLE IMAGES
plt.figure(figsize=(10,10))

for i, cls in enumerate(classes[:9]):
    class_path = os.path.join(dataset_path, cls)
    img_file = os.listdir(class_path)[0]
    img_path = os.path.join(class_path, img_file)

    img = Image.open(img_path)

    plt.subplot(3,3,i+1)
    plt.imshow(img)
    plt.title(cls)
    plt.axis('off')

plt.tight_layout()
plt.show()
```

Data Loading

Dataset yang digunakan pada proyek ini adalah **Animals-10 Dataset** yang diperoleh dari platform Kaggle: https://www.kaggle.com/datasets/antobenedetti/animals

Dataset dimuat dari Google Drive dan berhasil diekstrak ke direktori kerja. Berdasarkan eksplorasi awal, diperoleh informasi sebagai berikut:

- Total jumlah gambar: **26.179**
- Jumlah kelas: **10**
- Nama kelas:
  - *pecora, gatto, cavallo, gallina, mucca, elefante, ragno, scoiattolo, cane, farfalla*

Distribusi jumlah gambar pada tiap kelas bervariasi, dengan jumlah terbanyak pada kelas *cane* dan *ragno*, serta paling sedikit pada kelas *elefante*.

Selain itu, dataset memiliki **variasi resolusi gambar yang tinggi** (terdapat 162 variasi ukuran), yang menunjukkan bahwa data tidak seragam. Oleh karena itu, diperlukan tahap preprocessing seperti resizing agar seluruh gambar memiliki dimensi yang konsisten sebelum digunakan dalam proses training model.

Data Preprocessing

Split Dataset

```
# SPLIT DATASET
train_dir = "/content/dataset_split/train"
val_dir = "/content/dataset_split/val"
test_dir = "/content/dataset_split/test"

# Buat folder
for folder in [train_dir, val_dir, test_dir]:
    os.makedirs(folder, exist_ok=True)

# Split per class
for cls in classes:
    class_path = os.path.join(dataset_path, cls)
    images = os.listdir(class_path)

    train_imgs, temp_imgs = train_test_split(images, test_size=0.3,
random_state=42)
    val_imgs, test_imgs = train_test_split(temp_imgs, test_size=0.5,
random_state=42)

    # Buat subfolder
    os.makedirs(os.path.join(train_dir, cls), exist_ok=True)
    os.makedirs(os.path.join(val_dir, cls), exist_ok=True)
    os.makedirs(os.path.join(test_dir, cls), exist_ok=True)

    # Copy file
    for img in train_imgs:
        shutil.copy(os.path.join(class_path, img),
os.path.join(train_dir, cls))

    for img in val_imgs:
        shutil.copy(os.path.join(class_path, img),
os.path.join(val_dir, cls))

    for img in test_imgs:
        shutil.copy(os.path.join(class_path, img),
os.path.join(test_dir, cls))

print("Dataset berhasil di-split!")
```

```
Dataset berhasil di-split!
```

```python
# VERIFY SPLIT
def count_images(directory):
    total = 0
    for cls in os.listdir(directory):
        total += len(os.listdir(os.path.join(directory, cls)))
    return total

print("Train:", count_images(train_dir))
print("Validation:", count_images(val_dir))
print("Test:", count_images(test_dir))
```

```
Train: 18322
Validation: 3926
Test: 3931
```

```python
img_size = (224, 224)
batch_size = 64

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=30,
    zoom_range=0.3,
    horizontal_flip=True,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    fill_mode='nearest'
)

val_test_datagen = ImageDataGenerator(rescale=1./255)

# LOAD DATA WITH GENERATOR
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical'
)

val_generator = val_test_datagen.flow_from_directory(
    val_dir,
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical'
)

test_generator = val_test_datagen.flow_from_directory(
    test_dir,
    target_size=img_size,
```

```
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=False
)
```

```
Found 18322 images belonging to 10 classes.
Found 3926 images belonging to 10 classes.
Found 3931 images belonging to 10 classes.
```

```
# SAVE CLASS NAMES
class_names = list(train_generator.class_indices.keys())
print("Class Names:", class_names)
```

```
Class Names: ['cane', 'cavallo', 'elefante', 'farfalla', 'gallina',
'gatto', 'mucca', 'pecora', 'ragno', 'scoiattolo']
```

Data Preprocessing

Pada tahap ini, dataset dibagi menjadi tiga bagian utama, yaitu data latih (train), data validasi (validation), dan data uji (test). Pembagian ini bertujuan untuk memastikan model dapat dilatih dengan baik serta dievaluasi secara objektif terhadap data yang belum pernah dilihat sebelumnya.

Hasil pembagian dataset adalah sebagai berikut:

- Data latih (train): **18.322 gambar**
- Data validasi (validation): **3.926 gambar**
- Data uji (test): **3.931 gambar**

Total data tetap konsisten dan seluruh kelas terdistribusi ke dalam masing-masing subset.

Selanjutnya, dilakukan proses preprocessing menggunakan `ImageDataGenerator`, yang mencakup:

- Normalisasi pixel (rescale 1./255)
- Data augmentation pada data latih seperti rotasi, zoom, dan horizontal flip

Perlu diperhatikan bahwa **data augmentation hanya diterapkan pada data latih**, sedangkan data validasi dan data uji hanya dilakukan normalisasi. Hal ini bertujuan agar evaluasi model tetap merepresentasikan performa pada data asli.

Setelah preprocessing, dataset berhasil dimuat ke dalam generator dengan jumlah sebagai berikut:

- Train: 18.322 gambar (10 kelas)
- Validation: 3.926 gambar (10 kelas)
- Test: 3.931 gambar (10 kelas)

Adapun kelas yang digunakan dalam proses klasifikasi adalah: *cane, cavallo, elefante, farfalla, gallina, gatto, mucca, pecora, ragno, scoiattolo*

Modelling

□ MODEL 1: CNN PURE (STRONG)

```
model_cnn = Sequential([
    Conv2D(64, (3,3), activation='relu', padding='same',
input_shape=(224,224,3)),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(128, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(256, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(512, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Conv2D(512, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(1024, activation='relu'),
    Dropout(0.5),
    Dense(512, activation='relu'),
    Dropout(0.4),
    Dense(10, activation='softmax')
])

model_cnn.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

callbacks = [
    EarlyStopping(monitor='val_loss', patience=8,
restore_best_weights=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=4,
min_lr=1e-7, verbose=1),
    ModelCheckpoint('best_model_cnn.keras', monitor='val_accuracy',
save_best_only=True)
]

history_cnn = model_cnn.fit(
```

```
    train_generator,
    validation_data=val_generator,
    epochs=60,
    callbacks=callbacks
)
```

```
Epoch 1/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 142s 459ms/step - accuracy: 0.2647 -
loss: 2.3044 - val_accuracy: 0.1842 - val_loss: 5.3932 -
learning_rate: 1.0000e-04
Epoch 2/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 416ms/step - accuracy: 0.3563 -
loss: 1.8938 - val_accuracy: 0.4442 - val_loss: 1.6330 -
learning_rate: 1.0000e-04
Epoch 3/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 417ms/step - accuracy: 0.3968 -
loss: 1.7790 - val_accuracy: 0.5209 - val_loss: 1.3986 -
learning_rate: 1.0000e-04
Epoch 4/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 414ms/step - accuracy: 0.4336 -
loss: 1.6752 - val_accuracy: 0.5504 - val_loss: 1.3070 -
learning_rate: 1.0000e-04
Epoch 5/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 415ms/step - accuracy: 0.4721 -
loss: 1.5819 - val_accuracy: 0.5762 - val_loss: 1.2879 -
learning_rate: 1.0000e-04
Epoch 6/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 418ms/step - accuracy: 0.4966 -
loss: 1.5247 - val_accuracy: 0.5428 - val_loss: 1.3663 -
learning_rate: 1.0000e-04
Epoch 7/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 408ms/step - accuracy: 0.5121 -
loss: 1.4630 - val_accuracy: 0.6118 - val_loss: 1.1679 -
learning_rate: 1.0000e-04
Epoch 8/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 408ms/step - accuracy: 0.5324 -
loss: 1.3964 - val_accuracy: 0.5861 - val_loss: 1.2586 -
learning_rate: 1.0000e-04
Epoch 9/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 417ms/step - accuracy: 0.5514 -
loss: 1.3484 - val_accuracy: 0.6353 - val_loss: 1.0765 -
learning_rate: 1.0000e-04
Epoch 10/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 415ms/step - accuracy: 0.5740 -
loss: 1.2946 - val_accuracy: 0.5983 - val_loss: 1.1825 -
learning_rate: 1.0000e-04
Epoch 11/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 121s 422ms/step - accuracy: 0.5970 -
loss: 1.2143 - val_accuracy: 0.6454 - val_loss: 1.0876 -
learning_rate: 1.0000e-04
```

```
Epoch 12/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 418ms/step - accuracy: 0.6027 -
loss: 1.1860 - val_accuracy: 0.6816 - val_loss: 0.9712 -
learning_rate: 1.0000e-04
Epoch 13/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 419ms/step - accuracy: 0.6250 -
loss: 1.1341 - val_accuracy: 0.7203 - val_loss: 0.8943 -
learning_rate: 1.0000e-04
Epoch 14/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 415ms/step - accuracy: 0.6361 -
loss: 1.0968 - val_accuracy: 0.7155 - val_loss: 0.8923 -
learning_rate: 1.0000e-04
Epoch 15/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 412ms/step - accuracy: 0.6543 -
loss: 1.0536 - val_accuracy: 0.7084 - val_loss: 0.8882 -
learning_rate: 1.0000e-04
Epoch 16/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 413ms/step - accuracy: 0.6713 -
loss: 0.9898 - val_accuracy: 0.6994 - val_loss: 0.9133 -
learning_rate: 1.0000e-04
Epoch 17/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 410ms/step - accuracy: 0.6870 -
loss: 0.9650 - val_accuracy: 0.6544 - val_loss: 1.0973 -
learning_rate: 1.0000e-04
Epoch 18/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 410ms/step - accuracy: 0.6935 -
loss: 0.9389 - val_accuracy: 0.6673 - val_loss: 1.0709 -
learning_rate: 1.0000e-04
Epoch 19/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 121s 420ms/step - accuracy: 0.7020 -
loss: 0.9080 - val_accuracy: 0.7198 - val_loss: 0.8244 -
learning_rate: 1.0000e-04
Epoch 20/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 418ms/step - accuracy: 0.7106 -
loss: 0.8756 - val_accuracy: 0.7236 - val_loss: 0.8506 -
learning_rate: 1.0000e-04
Epoch 21/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 409ms/step - accuracy: 0.7266 -
loss: 0.8316 - val_accuracy: 0.7280 - val_loss: 0.8670 -
learning_rate: 1.0000e-04
Epoch 22/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 415ms/step - accuracy: 0.7376 -
loss: 0.7893 - val_accuracy: 0.7519 - val_loss: 0.8128 -
learning_rate: 1.0000e-04
Epoch 23/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 419ms/step - accuracy: 0.7442 -
loss: 0.7786 - val_accuracy: 0.8011 - val_loss: 0.6137 -
learning_rate: 1.0000e-04
Epoch 24/60
```

```
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 408ms/step - accuracy: 0.7554 -
loss: 0.7508 - val_accuracy: 0.7142 - val_loss: 0.8582 -
learning_rate: 1.0000e-04
Epoch 25/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 406ms/step - accuracy: 0.7660 -
loss: 0.7202 - val_accuracy: 0.7807 - val_loss: 0.6658 -
learning_rate: 1.0000e-04
Epoch 26/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 409ms/step - accuracy: 0.7708 -
loss: 0.7002 - val_accuracy: 0.7534 - val_loss: 0.7916 -
learning_rate: 1.0000e-04
Epoch 27/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 0s 390ms/step - accuracy: 0.7773 - loss:
0.6792
Epoch 27: ReduceLROnPlateau reducing learning rate to
1.9999999494757503e-05.
287/287 ━━━━━━━━━━━━━━━━━━━━ 116s 405ms/step - accuracy: 0.7775 -
loss: 0.6850 - val_accuracy: 0.7980 - val_loss: 0.6302 -
learning_rate: 1.0000e-04
Epoch 28/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 417ms/step - accuracy: 0.8064 -
loss: 0.5956 - val_accuracy: 0.8245 - val_loss: 0.5512 -
learning_rate: 2.0000e-05
Epoch 29/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 411ms/step - accuracy: 0.8197 -
loss: 0.5534 - val_accuracy: 0.8314 - val_loss: 0.5329 -
learning_rate: 2.0000e-05
Epoch 30/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 414ms/step - accuracy: 0.8201 -
loss: 0.5427 - val_accuracy: 0.8372 - val_loss: 0.5143 -
learning_rate: 2.0000e-05
Epoch 31/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 418ms/step - accuracy: 0.8237 -
loss: 0.5319 - val_accuracy: 0.8431 - val_loss: 0.4891 -
learning_rate: 2.0000e-05
Epoch 32/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 414ms/step - accuracy: 0.8334 -
loss: 0.5169 - val_accuracy: 0.8301 - val_loss: 0.5359 -
learning_rate: 2.0000e-05
Epoch 33/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 414ms/step - accuracy: 0.8304 -
loss: 0.5184 - val_accuracy: 0.8220 - val_loss: 0.5603 -
learning_rate: 2.0000e-05
Epoch 34/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 418ms/step - accuracy: 0.8360 -
loss: 0.4989 - val_accuracy: 0.8446 - val_loss: 0.4922 -
learning_rate: 2.0000e-05
Epoch 35/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 122s 426ms/step - accuracy: 0.8405 -
```

```
loss: 0.4879 - val_accuracy: 0.8566 - val_loss: 0.4729 -
learning_rate: 2.0000e-05
Epoch 36/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 419ms/step - accuracy: 0.8429 -
loss: 0.4832 - val_accuracy: 0.8426 - val_loss: 0.4887 -
learning_rate: 2.0000e-05
Epoch 37/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 412ms/step - accuracy: 0.8426 -
loss: 0.4810 - val_accuracy: 0.8444 - val_loss: 0.4892 -
learning_rate: 2.0000e-05
Epoch 38/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 411ms/step - accuracy: 0.8455 -
loss: 0.4689 - val_accuracy: 0.8263 - val_loss: 0.5368 -
learning_rate: 2.0000e-05
Epoch 39/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 0s 400ms/step - accuracy: 0.8550 - loss:
0.4541
Epoch 39: ReduceLROnPlateau reducing learning rate to
3.999999898951501e-06.
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 417ms/step - accuracy: 0.8529 -
loss: 0.4588 - val_accuracy: 0.8431 - val_loss: 0.4928 -
learning_rate: 2.0000e-05
Epoch 40/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 417ms/step - accuracy: 0.8574 -
loss: 0.4434 - val_accuracy: 0.8586 - val_loss: 0.4555 -
learning_rate: 4.0000e-06
Epoch 41/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 413ms/step - accuracy: 0.8599 -
loss: 0.4361 - val_accuracy: 0.8523 - val_loss: 0.4666 -
learning_rate: 4.0000e-06
Epoch 42/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 415ms/step - accuracy: 0.8580 -
loss: 0.4426 - val_accuracy: 0.8576 - val_loss: 0.4596 -
learning_rate: 4.0000e-06
Epoch 43/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 418ms/step - accuracy: 0.8582 -
loss: 0.4324 - val_accuracy: 0.8599 - val_loss: 0.4536 -
learning_rate: 4.0000e-06
Epoch 44/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 116s 406ms/step - accuracy: 0.8609 -
loss: 0.4202 - val_accuracy: 0.8607 - val_loss: 0.4562 -
learning_rate: 4.0000e-06
Epoch 45/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 116s 406ms/step - accuracy: 0.8593 -
loss: 0.4331 - val_accuracy: 0.8597 - val_loss: 0.4607 -
learning_rate: 4.0000e-06
Epoch 46/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 410ms/step - accuracy: 0.8584 -
loss: 0.4303 - val_accuracy: 0.8635 - val_loss: 0.4473 -
```

```
learning_rate: 4.0000e-06
Epoch 47/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 409ms/step - accuracy: 0.8652 -
loss: 0.4212 - val_accuracy: 0.8551 - val_loss: 0.4588 -
learning_rate: 4.0000e-06
Epoch 48/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 412ms/step - accuracy: 0.8636 -
loss: 0.4219 - val_accuracy: 0.8614 - val_loss: 0.4464 -
learning_rate: 4.0000e-06
Epoch 49/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 418ms/step - accuracy: 0.8674 -
loss: 0.4170 - val_accuracy: 0.8686 - val_loss: 0.4340 -
learning_rate: 4.0000e-06
Epoch 50/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 408ms/step - accuracy: 0.8606 -
loss: 0.4251 - val_accuracy: 0.8668 - val_loss: 0.4379 -
learning_rate: 4.0000e-06
Epoch 51/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 407ms/step - accuracy: 0.8685 -
loss: 0.4088 - val_accuracy: 0.8604 - val_loss: 0.4500 -
learning_rate: 4.0000e-06
Epoch 52/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 409ms/step - accuracy: 0.8646 -
loss: 0.4168 - val_accuracy: 0.8627 - val_loss: 0.4496 -
learning_rate: 4.0000e-06
Epoch 53/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 0s 385ms/step - accuracy: 0.8628 - loss:
0.4252
Epoch 53: ReduceLROnPlateau reducing learning rate to
7.999999979801942e-07.
287/287 ━━━━━━━━━━━━━━━━━━━━ 115s 401ms/step - accuracy: 0.8656 -
loss: 0.4173 - val_accuracy: 0.8635 - val_loss: 0.4406 -
learning_rate: 4.0000e-06
Epoch 54/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 116s 404ms/step - accuracy: 0.8664 -
loss: 0.4137 - val_accuracy: 0.8637 - val_loss: 0.4407 -
learning_rate: 8.0000e-07
Epoch 55/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 413ms/step - accuracy: 0.8656 -
loss: 0.4080 - val_accuracy: 0.8653 - val_loss: 0.4415 -
learning_rate: 8.0000e-07
Epoch 56/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 408ms/step - accuracy: 0.8675 -
loss: 0.4027 - val_accuracy: 0.8630 - val_loss: 0.4423 -
learning_rate: 8.0000e-07
Epoch 57/60
287/287 ━━━━━━━━━━━━━━━━━━━━ 0s 389ms/step - accuracy: 0.8668 - loss:
0.4030
Epoch 57: ReduceLROnPlateau reducing learning rate to
```

```
1.600000018697756e-07.
287/287 ━━━━━━━━━━━━━━━━━━━━ 116s 405ms/step - accuracy: 0.8653 -
loss: 0.4017 - val_accuracy: 0.8609 - val_loss: 0.4435 -
learning_rate: 8.0000e-07
Epoch 57: early stopping
Restoring model weights from the end of the best epoch: 49.
```

```
test_loss_cnn, test_acc_cnn = model_cnn.evaluate(test_generator)
print("CNN Test Accuracy:", test_acc_cnn)
```

```
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 115ms/step - accuracy: 0.8608 - loss:
0.4616
CNN Test Accuracy: 0.8608496785163879
```

Model 1: CNN

Model pertama yang digunakan adalah **Convolutional Neural Network (CNN)** dengan arsitektur **Sequential** yang terdiri dari layer **Conv2D**, **MaxPooling2D**, dan **Dense** untuk proses klasifikasi.

Selama proses training, model dilatih menggunakan data augmentation serta callback seperti EarlyStopping dan ReduceLROnPlateau untuk menjaga performa tetap optimal.

Berdasarkan hasil training:

- Akurasi training dan validasi berada di kisaran **86%–87%**
- Akurasi pada data uji (test set) sebesar **86.08%**

Hasil ini menunjukkan bahwa model CNN sudah mampu mengenali pola dasar pada gambar dengan cukup baik dan telah memenuhi kriteria akurasi minimum (>85%).

MODEL 2: CNN + TRANSFER LEARNING

```
base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)

base_model.trainable = False
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/mobilenet_v2/
mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
model_tl = Sequential([
    base_model,
    Conv2D(256, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    GlobalAveragePooling2D(),
    Dense(512, activation='relu'),
```

```
    Dropout(0.5),
    Dense(10, activation='softmax')
])

model_tl.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_tl = model_tl.fit(
    train_generator,
    validation_data=val_generator,
    epochs=20,
    callbacks=callbacks
)
```

```
Epoch 1/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 167s 515ms/step - accuracy: 0.8608 -
loss: 0.4890 - val_accuracy: 0.9508 - val_loss: 0.1661 -
learning_rate: 1.0000e-04
Epoch 2/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 116s 406ms/step - accuracy: 0.9078 -
loss: 0.2889 - val_accuracy: 0.9503 - val_loss: 0.1550 -
learning_rate: 1.0000e-04
Epoch 3/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 407ms/step - accuracy: 0.9170 -
loss: 0.2552 - val_accuracy: 0.9549 - val_loss: 0.1462 -
learning_rate: 1.0000e-04
Epoch 4/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 114s 399ms/step - accuracy: 0.9229 -
loss: 0.2331 - val_accuracy: 0.9531 - val_loss: 0.1441 -
learning_rate: 1.0000e-04
Epoch 5/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 115s 399ms/step - accuracy: 0.9260 -
loss: 0.2232 - val_accuracy: 0.9557 - val_loss: 0.1386 -
learning_rate: 1.0000e-04
Epoch 6/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 114s 396ms/step - accuracy: 0.9311 -
loss: 0.2098 - val_accuracy: 0.9539 - val_loss: 0.1382 -
learning_rate: 1.0000e-04
Epoch 7/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 407ms/step - accuracy: 0.9356 -
loss: 0.1947 - val_accuracy: 0.9570 - val_loss: 0.1407 -
learning_rate: 1.0000e-04
Epoch 8/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 409ms/step - accuracy: 0.9396 -
loss: 0.1875 - val_accuracy: 0.9534 - val_loss: 0.1418 -
learning_rate: 1.0000e-04
Epoch 9/20
```

```
287/287 ━━━━━━━━━━━━━━━━━━━━ 115s 401ms/step - accuracy: 0.9378 -
loss: 0.1884 - val_accuracy: 0.9562 - val_loss: 0.1358 -
learning_rate: 1.0000e-04
Epoch 10/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 116s 404ms/step - accuracy: 0.9405 -
loss: 0.1780 - val_accuracy: 0.9559 - val_loss: 0.1371 -
learning_rate: 1.0000e-04
Epoch 11/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 115s 401ms/step - accuracy: 0.9462 -
loss: 0.1670 - val_accuracy: 0.9575 - val_loss: 0.1278 -
learning_rate: 1.0000e-04
Epoch 12/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 407ms/step - accuracy: 0.9466 -
loss: 0.1581 - val_accuracy: 0.9531 - val_loss: 0.1423 -
learning_rate: 1.0000e-04
Epoch 13/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 410ms/step - accuracy: 0.9473 -
loss: 0.1600 - val_accuracy: 0.9585 - val_loss: 0.1341 -
learning_rate: 1.0000e-04
Epoch 14/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 408ms/step - accuracy: 0.9510 -
loss: 0.1465 - val_accuracy: 0.9516 - val_loss: 0.1525 -
learning_rate: 1.0000e-04
Epoch 15/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 0s 394ms/step - accuracy: 0.9508 - loss:
0.1443
Epoch 15: ReduceLROnPlateau reducing learning rate to
1.9999999494757503e-05.
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 410ms/step - accuracy: 0.9508 -
loss: 0.1471 - val_accuracy: 0.9564 - val_loss: 0.1404 -
learning_rate: 1.0000e-04
Epoch 16/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 415ms/step - accuracy: 0.9570 -
loss: 0.1254 - val_accuracy: 0.9592 - val_loss: 0.1365 -
learning_rate: 2.0000e-05
Epoch 17/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 409ms/step - accuracy: 0.9594 -
loss: 0.1222 - val_accuracy: 0.9590 - val_loss: 0.1342 -
learning_rate: 2.0000e-05
Epoch 18/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 413ms/step - accuracy: 0.9624 -
loss: 0.1185 - val_accuracy: 0.9582 - val_loss: 0.1327 -
learning_rate: 2.0000e-05
Epoch 19/20
287/287 ━━━━━━━━━━━━━━━━━━━━ 0s 392ms/step - accuracy: 0.9598 - loss:
0.1249
Epoch 19: ReduceLROnPlateau reducing learning rate to
3.999999898951501e-06.
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 409ms/step - accuracy: 0.9615 -
```

```
loss: 0.1192 - val_accuracy: 0.9577 - val_loss: 0.1336 -
learning_rate: 2.0000e-05
Epoch 19: early stopping
Restoring model weights from the end of the best epoch: 11.
```

```
base_model.trainable = True
for layer in base_model.layers[:-50]:
    layer.trainable = False

model_tl.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_tl_fine = model_tl.fit(
    train_generator,
    validation_data=val_generator,
    epochs=25,
    callbacks=callbacks
)
```

```
Epoch 1/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 163s 499ms/step - accuracy: 0.9185 -
loss: 0.2397 - val_accuracy: 0.9598 - val_loss: 0.1296 -
learning_rate: 1.0000e-05
Epoch 2/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 409ms/step - accuracy: 0.9349 -
loss: 0.1958 - val_accuracy: 0.9623 - val_loss: 0.1213 -
learning_rate: 1.0000e-05
Epoch 3/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 413ms/step - accuracy: 0.9406 -
loss: 0.1842 - val_accuracy: 0.9610 - val_loss: 0.1188 -
learning_rate: 1.0000e-05
Epoch 4/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 116s 405ms/step - accuracy: 0.9447 -
loss: 0.1704 - val_accuracy: 0.9618 - val_loss: 0.1149 -
learning_rate: 1.0000e-05
Epoch 5/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 407ms/step - accuracy: 0.9474 -
loss: 0.1583 - val_accuracy: 0.9620 - val_loss: 0.1154 -
learning_rate: 1.0000e-05
Epoch 6/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 415ms/step - accuracy: 0.9522 -
loss: 0.1440 - val_accuracy: 0.9646 - val_loss: 0.1130 -
learning_rate: 1.0000e-05
Epoch 7/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 413ms/step - accuracy: 0.9539 -
loss: 0.1413 - val_accuracy: 0.9631 - val_loss: 0.1147 -
learning_rate: 1.0000e-05
```

```
Epoch 8/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 405ms/step - accuracy: 0.9564 -
loss: 0.1304 - val_accuracy: 0.9643 - val_loss: 0.1129 -
learning_rate: 1.0000e-05
Epoch 9/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 413ms/step - accuracy: 0.9580 -
loss: 0.1304 - val_accuracy: 0.9641 - val_loss: 0.1131 -
learning_rate: 1.0000e-05
Epoch 10/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 412ms/step - accuracy: 0.9584 -
loss: 0.1263 - val_accuracy: 0.9648 - val_loss: 0.1109 -
learning_rate: 1.0000e-05
Epoch 11/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 415ms/step - accuracy: 0.9618 -
loss: 0.1138 - val_accuracy: 0.9648 - val_loss: 0.1125 -
learning_rate: 1.0000e-05
Epoch 12/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 413ms/step - accuracy: 0.9623 -
loss: 0.1114 - val_accuracy: 0.9651 - val_loss: 0.1116 -
learning_rate: 1.0000e-05
Epoch 13/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 419ms/step - accuracy: 0.9638 -
loss: 0.1102 - val_accuracy: 0.9656 - val_loss: 0.1083 -
learning_rate: 1.0000e-05
Epoch 14/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 407ms/step - accuracy: 0.9677 -
loss: 0.0995 - val_accuracy: 0.9648 - val_loss: 0.1078 -
learning_rate: 1.0000e-05
Epoch 15/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 408ms/step - accuracy: 0.9668 -
loss: 0.0998 - val_accuracy: 0.9656 - val_loss: 0.1056 -
learning_rate: 1.0000e-05
Epoch 16/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 413ms/step - accuracy: 0.9699 -
loss: 0.0974 - val_accuracy: 0.9648 - val_loss: 0.1074 -
learning_rate: 1.0000e-05
Epoch 17/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 119s 415ms/step - accuracy: 0.9695 -
loss: 0.0961 - val_accuracy: 0.9641 - val_loss: 0.1085 -
learning_rate: 1.0000e-05
Epoch 18/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 118s 413ms/step - accuracy: 0.9706 -
loss: 0.0923 - val_accuracy: 0.9646 - val_loss: 0.1097 -
learning_rate: 1.0000e-05
Epoch 19/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 0s 403ms/step - accuracy: 0.9750 - loss:
0.0801
Epoch 19: ReduceLROnPlateau reducing learning rate to
1.9999999494757505e-06.
```

```
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 419ms/step - accuracy: 0.9731 -
loss: 0.0854 - val_accuracy: 0.9656 - val_loss: 0.1087 -
learning_rate: 1.0000e-05
Epoch 20/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 409ms/step - accuracy: 0.9742 -
loss: 0.0820 - val_accuracy: 0.9656 - val_loss: 0.1069 -
learning_rate: 2.0000e-06
Epoch 21/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 117s 408ms/step - accuracy: 0.9732 -
loss: 0.0798 - val_accuracy: 0.9654 - val_loss: 0.1070 -
learning_rate: 2.0000e-06
Epoch 22/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 120s 419ms/step - accuracy: 0.9743 -
loss: 0.0780 - val_accuracy: 0.9641 - val_loss: 0.1069 -
learning_rate: 2.0000e-06
Epoch 23/25
287/287 ━━━━━━━━━━━━━━━━━━━━ 0s 389ms/step - accuracy: 0.9744 - loss:
0.0782
Epoch 23: ReduceLROnPlateau reducing learning rate to
3.999999989900971e-07.
287/287 ━━━━━━━━━━━━━━━━━━━━ 116s 404ms/step - accuracy: 0.9744 -
loss: 0.0811 - val_accuracy: 0.9654 - val_loss: 0.1077 -
learning_rate: 2.0000e-06
Epoch 23: early stopping
Restoring model weights from the end of the best epoch: 15.
```

```
test_loss_tl, test_acc_tl = model_tl.evaluate(test_generator)
print("Transfer Learning Test Accuracy:", test_acc_tl)
```

```
62/62 ━━━━━━━━━━━━━━━━━━━━ 17s 276ms/step - accuracy: 0.9646 - loss:
0.1235
Transfer Learning Test Accuracy: 0.9646400213241577
```

[] Model 2: Transfer Learning (MobileNetV2)

Pada model kedua, digunakan pendekatan **Transfer Learning** dengan memanfaatkan **MobileNetV2** sebagai feature extractor, kemudian ditambahkan beberapa layer seperti **Conv2D**, **GlobalAveragePooling2D**, dan **Dense** untuk proses klasifikasi.

Proses training dilakukan dalam dua tahap, yaitu:

1. **Feature extraction (freeze)** untuk melatih layer tambahan
2. **Fine-tuning** untuk meningkatkan performa model

Berdasarkan hasil training:

- Akurasi training mencapai sekitar **97%**
- Akurasi validasi mencapai sekitar **96%–97%**
- Akurasi pada data uji (test set) sebesar **96.46%**

Hasil ini menunjukkan bahwa model mampu mengenali pola gambar dengan sangat baik dan mengalami peningkatan performa yang signifikan dibandingkan model CNN sebelumnya.

PERBANDINGAN MODEL

```
print("\n=== PERBANDINGAN MODEL ===")
print(f"CNN Accuracy: {test_acc_cnn:.4f}")
print(f"Transfer Learning Accuracy: {test_acc_tl:.4f}")
```

```
=== PERBANDINGAN MODEL ===
CNN Accuracy: 0.8608
Transfer Learning Accuracy: 0.9646
```

Perbandingan Model

Setelah dilakukan pelatihan dan evaluasi, diperoleh hasil sebagai berikut:

- **CNN Accuracy:** 86.08%
- **Transfer Learning Accuracy:** 96.46%

Model **CNN** mampu mencapai akurasi yang cukup baik dan telah memenuhi kriteria minimum. Namun, model ini masih memiliki keterbatasan dalam menangkap fitur yang lebih kompleks.

Sementara itu, model **Transfer Learning** menunjukkan performa yang jauh lebih tinggi dengan akurasi di atas 96%. Hal ini disebabkan oleh penggunaan model pretrained (MobileNetV2) yang sudah dilatih pada dataset besar, sehingga mampu mengekstraksi fitur gambar dengan lebih optimal.

Berdasarkan hasil tersebut, **model Transfer Learning dipilih sebagai model terbaik (final_model)** karena memiliki performa yang lebih unggul dalam melakukan klasifikasi gambar.

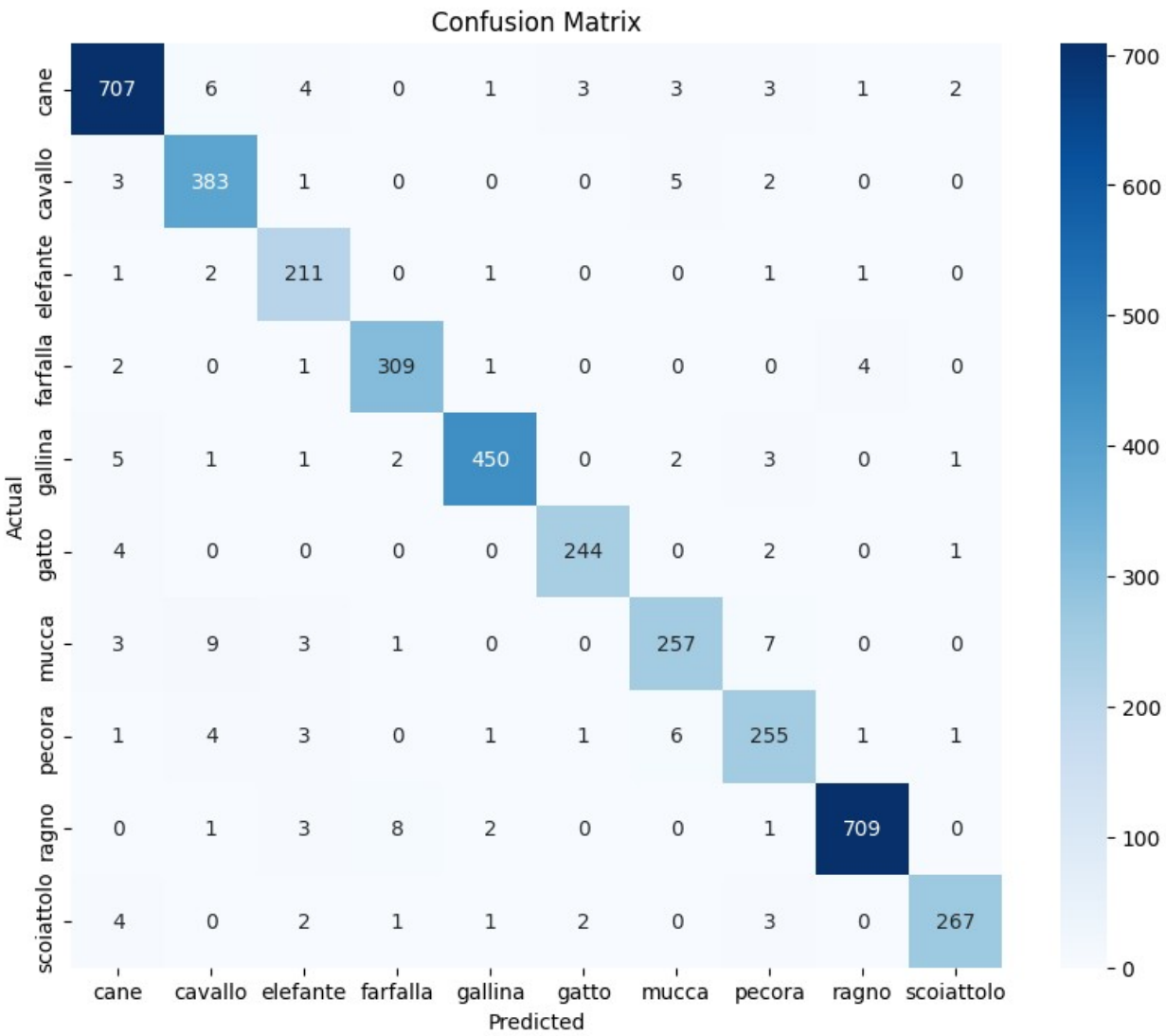
Evaluasi dan Visualisasi

```
# === PREDICTION ===
y_pred = final_model.predict(test_generator)
y_pred_classes = np.argmax(y_pred, axis=1)

y_true = test_generator.classes
class_labels = list(test_generator.class_indices.keys())

# === CLASSIFICATION REPORT ===
from sklearn.metrics import classification_report
print("Classification Report:\n")
print(classification_report(y_true, y_pred_classes,
target_names=class_labels))
```

```
62/62 ━━━━━━━━━━━━━━━━━━━━ 16s 163ms/step
Classification Report:

              precision    recall  f1-score   support
```

```
        cane       0.97      0.97      0.97       730
     cavallo       0.94      0.97      0.96       394
    elefante       0.92      0.97      0.95       217
    farfalla       0.96      0.97      0.97       317
     gallina       0.98      0.97      0.98       465
       gatto       0.98      0.97      0.97       251
       mucca       0.94      0.92      0.93       280
      pecora       0.92      0.93      0.93       273
       ragno       0.99      0.98      0.98       724
  scoiattolo       0.98      0.95      0.97       280

    accuracy                           0.96      3931
   macro avg       0.96      0.96      0.96      3931
weighted avg       0.96      0.96      0.96      3931
```

```python
# === CONFUSION MATRIX ===
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_true, y_pred_classes)

plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_labels,
            yticklabels=class_labels)

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```




```
final_model.evaluate(test_generator)
```

```
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 68ms/step - accuracy: 0.9646 - loss:
0.1235
```

```
[0.12351193279027939, 0.9646400213241577]
```

Evaluasi dan Visualisasi

Evaluasi model dilakukan menggunakan **confusion matrix** dan **classification report** untuk melihat performa klasifikasi pada setiap kelas.

Berdasarkan hasil evaluasi:

- Akurasi keseluruhan mencapai **96.46%**
- Nilai **precision, recall, dan f1-score** untuk sebagian besar kelas berada di atas **0.95**
- Model mampu mengklasifikasikan hampir seluruh kelas dengan sangat baik

Dari confusion matrix, terlihat bahwa prediksi model didominasi oleh nilai diagonal yang tinggi, menandakan bahwa sebagian besar prediksi sudah tepat. Kesalahan klasifikasi relatif kecil dan hanya terjadi pada beberapa kelas yang memiliki kemiripan visual.

Secara keseluruhan, model menunjukkan performa yang sangat baik dan mampu melakukan generalisasi dengan baik terhadap data uji.

Konversi Model

```
# SAVE MODEL (SavedModel)
final_model.export("saved_model")

print("Model berhasil disimpan dalam format SavedModel")
```

```
Saved artifact at 'saved_model'. The following endpoints are
available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3),
dtype=tf.float32, name='keras_tensor_182')
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
  135698719850320: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698719849936: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698719849168: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698719849552: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698719850128: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822120656: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822132176: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822133136: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822134672: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822134480: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822132944: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822135056: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822133520: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822134864: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822132752: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698822134288: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286167632: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286167824: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286167056: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286166480: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286166288: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286167440: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286167248: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286166864: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286168784: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286168208: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698286168400: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
135698286168592: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286166096: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286169744: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286169168: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286169360: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286169552: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286166672: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286170704: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286170128: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286170320: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286170512: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286168016: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286171664: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286171088: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286171280: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286171472: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286168976: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286172624: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286172048: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286172240: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286172432: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286169936: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286173584: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286173008: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286173200: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286173392: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286170896: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286174544: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286173968: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286174160: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286174352: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286171856: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286175504: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286174928: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286175120: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286175312: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286172816: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286176464: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286175888: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286176080: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286176272: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286173776: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286177424: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286176848: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286177040: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286177232: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286174736: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286178384: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286177808: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
135698286178000: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286178192: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286175696: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286179344: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286178768: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286178960: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286179152: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286176656: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286180304: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286179728: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286179920: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286180112: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286177616: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286181264: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286180688: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286180880: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286181648: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286178576: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286182224: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286181072: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286181840: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286179536: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286182032: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286181456: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698286180496: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284856720: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284856336: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698719848976: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284857104: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284855760: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284856912: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284855952: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284856144: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284858064: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284857488: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284857680: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284857872: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284855568: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284859024: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284858448: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284858640: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284858832: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284855376: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284859984: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284859408: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284859600: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284859792: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284857296: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284860944: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
135698284860368: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284860560: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284860752: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284858256: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284861904: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284861328: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284861520: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284861712: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284859216: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284862864: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284862288: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284862480: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284862672: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284860176: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284863824: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284863248: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284863440: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284863632: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284861136: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284864784: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284864208: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284864400: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284864592: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284862096: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284865744: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284865168: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284865360: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284865552: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284863056: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284866704: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284866128: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284866320: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284866512: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284864016: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284867664: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284867088: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284867280: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284867472: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284864976: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284868624: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284868048: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284868240: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284868432: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284865936: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284869584: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284869008: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284869200: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284869392: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284866896: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
135698284870544: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284869968: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284870160: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284870928: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284867856: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284871504: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284870352: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284871120: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284868816: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284871312: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284870736: TensorSpec(shape=(), dtype=tf.resource, name=None)
135698284869776: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754211920: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754212112: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754211344: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754210768: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754210576: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754211728: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754211536: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754211152: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754213072: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754212496: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754212688: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754212880: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754210384: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754214032: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754213456: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754213648: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754213840: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754210960: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754214992: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754214416: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754214608: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754214800: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754212304: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754215952: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754215376: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754215568: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754215760: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754213264: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754216912: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754216336: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754216528: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754216720: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754214224: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754217872: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754217296: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754217488: TensorSpec(shape=(), dtype=tf.resource, name=None)
135697754217680: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
  135697754215184: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754218832: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754218256: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754218448: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754218640: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754216144: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754219792: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754219216: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754219408: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754219600: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754217104: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754220752: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754220176: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754220368: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754220560: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754218064: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754221712: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754221136: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754221328: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754221520: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754219024: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754222672: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754222096: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754222288: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754222480: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754219984: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754223632: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754223056: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754223248: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754223440: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754220944: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754224592: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754224016: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754224208: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754224400: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754221904: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754225552: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135698719849360: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754226512: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754224784: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754226128: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754222864: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754223824: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754226320: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697754225744: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697753163152: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135697753162192: TensorSpec(shape=(), dtype=tf.resource, name=None)
Model berhasil disimpan dalam format SavedModel
```

```python
# CONVERT TO TFLITE
import tensorflow as tf

converter = tf.lite.TFLiteConverter.from_saved_model("saved_model")
tflite_model = converter.convert()

# Simpan file
with open("model.tflite", "wb") as f:
    f.write(tflite_model)

print("Model berhasil dikonversi ke TFLite")
```

```
Model berhasil dikonversi ke TFLite
```

```python
# SAVE LABEL
with open("label.txt", "w") as f:
    for label in class_names:
        f.write(label + "\n")

print("Label berhasil disimpan")
```

```
Label berhasil disimpan
```

```python
# CONVERT TO TFJS
!tensorflowjs_converter \
  --input_format=tf_saved_model \
  saved_model \
  tfjs_model

print("Model berhasil dikonversi ke TFJS")
```

```
2026-04-27 16:01:11.621348: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to
register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
E0000 00:00:1777305671.636332   64096 cuda_dnn.cc:8579] Unable to
register cuDNN factory: Attempting to register factory for plugin
cuDNN when one has already been registered
E0000 00:00:1777305671.640940   64096 cuda_blas.cc:1407] Unable to
register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered
W0000 00:00:1777305671.652206   64096 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.
W0000 00:00:1777305671.652229   64096 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.
W0000 00:00:1777305671.652233   64096 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
```

```
linking the same target more than once.
W0000 00:00:1777305671.652234   64096 computation_placer.cc:177]
computation placer already registered. Please check linkage and avoid
linking the same target more than once.
 Try https://ydf.readthedocs.io, the successor of TensorFlow Decision
Forests with more features and faster training!
2026-04-27 16:01:15.638981: W
tensorflow/core/common_runtime/gpu/gpu_bfc_allocator.cc:47] Overriding
orig_value setting because the TF_FORCE_GPU_ALLOW_GROWTH environment
variable is set. Original config value was 0.
I0000 00:00:1777305675.639973   64096 gpu_device.cc:2019] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 61156 MB
memory:  -> device: 0, name: NVIDIA H100 80GB HBM3, pci bus id:
0000:04:00.0, compute capability: 9.0
I0000 00:00:1777305676.391379   64096 devices.cc:67] Number of
eligible GPUs (core count >= 8, compute capability >= 0.0): 1
WARNING: All log messages before absl::InitializeLog() is called are
written to STDERR
I0000 00:00:1777305676.391529   64096 single_machine.cc:374] Starting
new session
I0000 00:00:1777305676.396501   64096 gpu_device.cc:2019] Created
device /job:localhost/replica:0/task:0/device:GPU:0 with 61156 MB
memory:  -> device: 0, name: NVIDIA H100 80GB HBM3, pci bus id:
0000:04:00.0, compute capability: 9.0
Model berhasil dikonversi ke TFJS
```

```python
# ORGANIZE SUBMISSION FOLDER

# Buat folder
os.makedirs("submission/tflite", exist_ok=True)
os.makedirs("submission/tfjs_model", exist_ok=True)
os.makedirs("submission/saved_model", exist_ok=True)

# Pindahkan file
shutil.move("model.tflite", "submission/tflite/model.tflite")
shutil.move("label.txt", "submission/tflite/label.txt")

shutil.copytree("saved_model", "submission/saved_model",
dirs_exist_ok=True)
shutil.copytree("tfjs_model", "submission/tfjs_model",
dirs_exist_ok=True)

print("Struktur submission siap!")
```

```
Struktur submission siap!
```

```python
shutil.make_archive("submission_final", 'zip', "submission")

print("ZIP siap untuk dikumpulkan!")
```

```
ZIP siap untuk dikumpulkan!
```

Konversi Model

Setelah model selesai dilatih, dilakukan proses konversi model ke beberapa format agar dapat digunakan di berbagai platform.

Model berhasil dikonversi ke tiga format utama, yaitu:

- **SavedModel** → digunakan untuk deployment berbasis TensorFlow
- **TensorFlow Lite (TFLite)** → digunakan untuk perangkat mobile dan embedded system
- **TensorFlow.js (TFJS)** → digunakan untuk aplikasi berbasis web

Selain itu, file label juga disimpan untuk memetakan hasil prediksi ke nama kelas.

Seluruh proses konversi berhasil dilakukan tanpa kendala, dan struktur folder submission telah disusun sesuai dengan ketentuan yang diberikan.

Inference (Optional)

```
import numpy as np
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt

# Ambil 1 gambar dari test set
img_path = test_generator.filepaths[0]

# Load image
img = image.load_img(img_path, target_size=(224,224))
img_array = image.img_to_array(img) / 255.0
img_array = np.expand_dims(img_array, axis=0)

# Predict
prediction = final_model.predict(img_array)
predicted_class = class_names[np.argmax(prediction)]
confidence = np.max(prediction)

# Show result
plt.imshow(img)
plt.title(f"Predicted: {predicted_class}\nConfidence:
{confidence:.2f}")
plt.axis('off')
plt.show()

print("Prediction:", predicted_class)
print("Confidence:", confidence)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 9s 9s/step
```

```
Prediction: cane
Confidence: 0.99718034
```

```python
# MULTI INFERENCE
for i in range(5):
    img_path = test_generator.filepaths[i]

    img = image.load_img(img_path, target_size=(224,224))
    img_array = image.img_to_array(img) / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    pred = final_model.predict(img_array)
    predicted_class = class_names[np.argmax(pred)]

    plt.imshow(img)
    plt.title(f"Pred: {predicted_class}")
    plt.axis('off')
    plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
```

Pred: cane

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
```

Pred: cane

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
```

Pred: cane

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
```

Pred: cane

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
```

Pred: cane

Inference

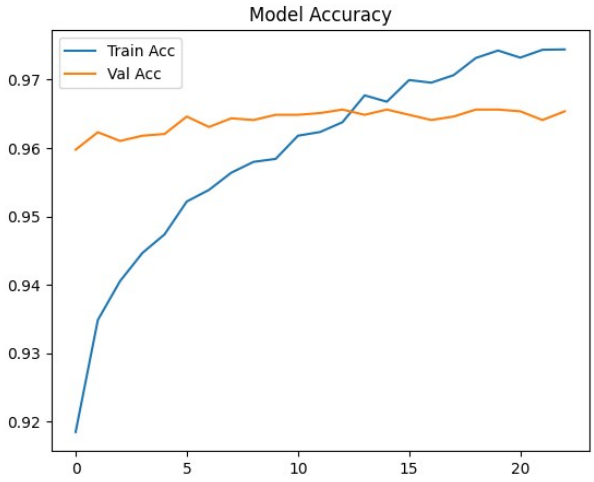
Model yang telah dilatih kemudian digunakan untuk melakukan prediksi terhadap beberapa gambar dari data uji (test set).

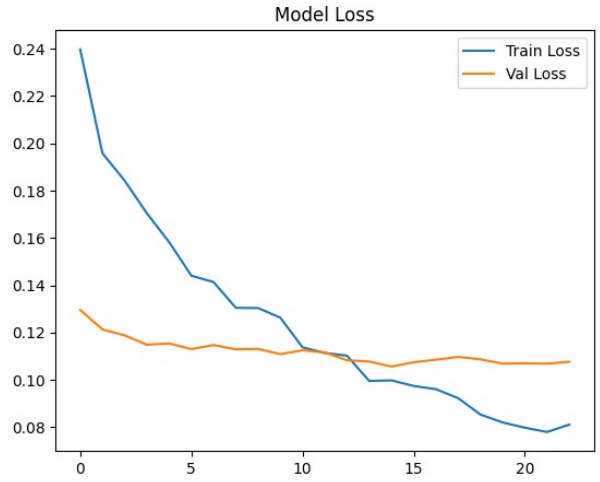
Hasil inference menunjukkan bahwa model mampu mengklasifikasikan gambar dengan sangat baik. Pada contoh yang ditampilkan, seluruh gambar berhasil diprediksi sebagai kelas *cane* dengan tingkat kepercayaan (confidence) yang sangat tinggi, bahkan mendekati 1.00.

Hal ini menunjukkan bahwa model tidak hanya memiliki akurasi yang tinggi secara numerik, tetapi juga mampu memberikan prediksi yang konsisten pada data baru yang belum pernah dilihat sebelumnya.

Dengan demikian, model dapat dikatakan memiliki kemampuan generalisasi yang baik dan siap digunakan untuk kasus klasifikasi gambar serupa.

```
if test_acc_tl > test_acc_cnn:
    history_used = history_tl_fine
else:
    history_used = history_cnn

hist = history_used.history
acc = hist['accuracy']
val_acc = hist['val_accuracy']
loss = hist['loss']
val_loss = hist['val_loss']

plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
plt.plot(acc, label='Train Acc')
plt.plot(val_acc, label='Val Acc')
plt.title('Model Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(loss, label='Train Loss')
plt.plot(val_loss, label='Val Loss')
plt.title('Model Loss')
plt.legend()
plt.show()
```


Model Accuracy
Train Acc
Val Acc
0.97
0.96
0.95
0.94
0.93
0.92
0
5
10
15
20


Model Loss
Train Loss
Val Loss
0.24
0.22
0.20
0.18
0.16
0.14
0.12
0.10
0.08
0
5
10
15
20